

# Documentação Técnica - Sistema de Emissão de Notas Fiscais

## 1. Visão Geral

Este documento descreve a solução desenvolvida para o desafio técnico da KORP. O objetivo é apresentar um sistema de emissão de notas fiscais construído utilizando uma arquitetura de microsserviços, focando em simplicidade, código limpo, alta disponibilidade e resiliência (tratamento de falhas).

## 2. Arquitetura da Solução

O sistema foi dividido em três camadas principais, garantindo o desacoplamento das responsabilidades:

- **Frontend:** Desenvolvido em Angular (executando na porta 4200), utilizando componentes standalone e consumo reativo de APIs REST.
- **Backend (Microsserviços):** Desenvolvido em Golang com dois serviços independentes: Estoque (porta 8081) e Faturamento (porta 8082), ambos equipados com tratamento de CORS e persistência resiliente.
- **Banco de Dados & Persistência:** PostgreSQL gerenciando os dados de forma relacional, com um mecanismo automático de fallback para armazenamento thread-safe em memória caso o banco de dados esteja indisponível.

## 3. Detalhamento dos Microsserviços

Abaixo estão as responsabilidades e endpoints principais de cada serviço construído no backend:

| Microsserviço      | Porta | Responsabilidade                                | Endpoints Principais                                                                                                                                                               |
|--------------------|-------|-------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serviço de Estoque | 8081  | Controle de produtos e gerenciamento de saldos. | <ul style="list-style-type: none"><li>• POST /produtos (Cadastro/Atualização)</li><li>• GET /produtos (Listagem)</li><li>• POST /produtos/{id}/baixar (Baixa de Estoque)</li></ul> |

| Microserviço           | Porta | Responsabilidade                                          | Endpoints Principais                                                                                                                                                                          |
|------------------------|-------|-----------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Serviço de Faturamento | 8082  | Criação de notas fiscais, listagem e rotina de impressão. | <ul style="list-style-type: none"> <li>• POST /notas (Emissão de Nota)</li> <li>• GET /notas (Listagem de Notas)</li> <li>• POST /notas/{id}/imprimir (Processamento e Fechamento)</li> </ul> |

## 4. Detalhamento Técnico - Frontend (Angular)

### Ciclos de Vida Utilizados

- **ngOnInit:** Utilizado na inicialização dos componentes para requisições de carga de dados iniciais (listagens de produtos e notas fiscais em tempo real).
- **ngOnDestroy:** Utilizado para realizar o "unsubscribe" de Observables abertos e evitar vazamentos de memória (memory leaks).

### Comunicação HTTP e Tratamento de Estados

- **HttpClientModule & FormsModule:** Utilizados para binding bidirecional de formulários e requisições HTTP REST assíncronas.
- **Feedback ao Usuário e Prevenção de Duplo Clique:** Durante a impressão de notas fiscais, o sistema desabilita visualmente o botão e altera o rótulo para "Processando Impressão..." (carregando), evitando requisições duplicadas e fornecendo clareza sobre o estado da transação.
- **Tratamento de Erros com RxJS:** Uso do operador catchError para capturar falhas HTTP (como indisponibilidade do serviço de estoque) e exibir mensagens informativas ao usuário.

### Bibliotecas Visuais e Auxiliares

- **Angular Material:** Fornece a base de componentes de UI, como indicadores de carregamento (spinners) para bloquear a tela durante o processamento de impressões.
- **ng-toastr / MatSnackBar:** Empregado para dar feedbacks visuais rápidos (sucesso ou erro) ao usuário.

## 5. Detalhamento Técnico - Backend (Golang)

### Middleware de Suporte a CORS

Para autorizar a integração entre o frontend Angular (<http://localhost:4200>) e os microsserviços em Go (portas 8081 e 8082), foi implementado o middleware enableCORS com injeção dos cabeçalhos HTTP necessários e tratamento nativo para requisições Preflight do tipo OPTIONS (retornando status 200 OK).

### Gerenciamento de Dependências, Frameworks e Persistência

- **Go Modules:** Gerenciamento nativo de dependências através dos arquivos go.mod e go.sum independentes em cada microsserviço.
- **Gorilla Mux:** Escolhido pela simplicidade na criação de rotas semânticas, extração de variáveis de URL e acoplamento de middlewares.
- **database/sql & lib/pq:** Combinação padrão e performática para manipular instruções SQL no banco de dados PostgreSQL.
- **Resiliência com Fallback em Memória (sync.Mutex):** Na inicialização (initDB()), o sistema testa a conexão com o PostgreSQL. Caso o banco esteja offline, a aplicação chaveia automaticamente para um repositório em memória utilizando mapas Go e mutexes thread-safe, garantindo execução sem travamentos.

*Nota: Este projeto não utiliza as linguagens C# ou a biblioteca LINQ, sendo 100% desenvolvido em Golang no backend.*

## 6. Resiliência e Tratamento de Falhas

A arquitetura de microsserviços exige cuidado na comunicação entre aplicações. Neste projeto, o requisito obrigatório de falha é tratado da seguinte maneira durante a **Impressão de Notas Fiscais**:

1. O usuário solicita a impressão através do Frontend.
2. O Serviço de Faturamento recebe o pedido, valida se a nota está "Aberta", e faz uma requisição HTTP REST para o Serviço de Estoque solicitando a baixa dos itens.
3. **Cenário de Falha de Comunicação:** Caso o Serviço de Estoque esteja fora do ar ou o saldo seja insuficiente, a requisição é interceptada.
4. **Recuperação & Consistência:** A transação de atualização do status da nota fiscal é cancelada. A nota permanece no status original ("Aberta"), garantindo a consistência dos dados.
5. **Indisponibilidade do Banco:** Se o PostgreSQL cair durante a execução, o fallback em memória assume a transação sem interromper o serviço.
6. O usuário recebe no frontend uma mensagem clara sobre a falha de comunicação entre os sistemas.